
TÉCNICAS DE APRENDIZADO DE MÁQUINA

Bacharelado em Sistemas da Informação

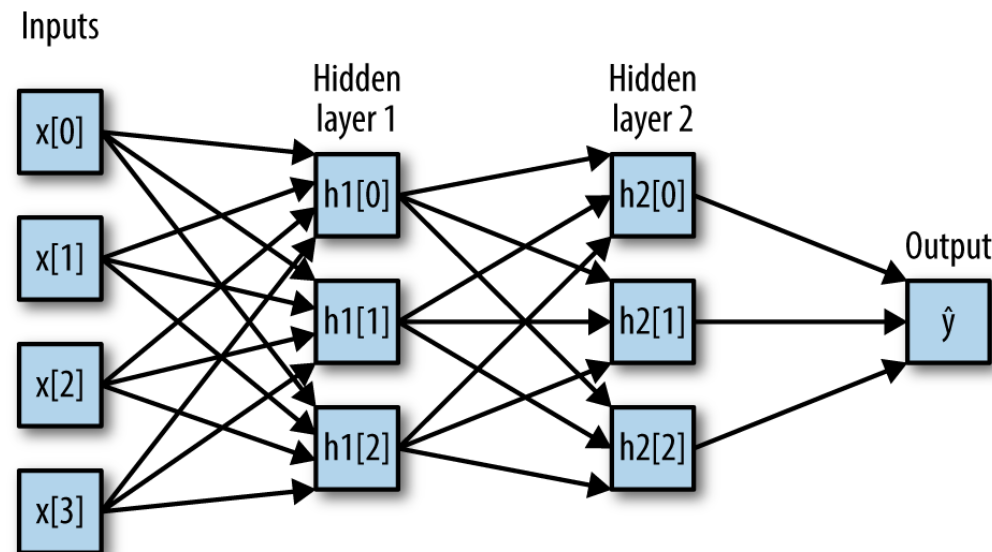
Prof. Marco André Abud Kappel

Aula 13 – Regressão com Redes Neurais

Regressão com Redes Neurais

- **Revisão**

- Um dos tipos mais comuns de Redes Neurais é formada por uma composição de várias **camadas** de Perceptrons.
- Neste caso, temos muito mais **coeficientes** para serem aprendidos.

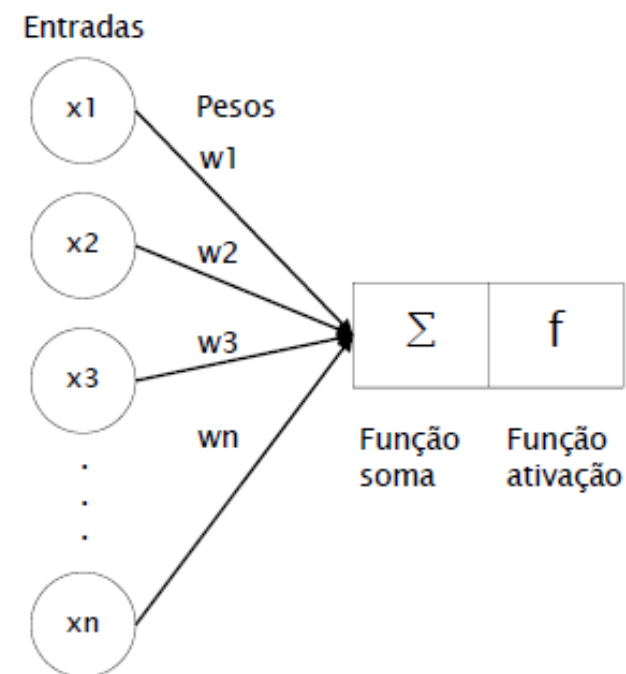


Regressão com Redes Neurais

- **Revisão**

- Funcionamento:

- O neurônio recebe diversas informações como **entrada**.
 - Cada entrada é multiplicada por um **peso**.
 - As entradas são combinadas por uma **função soma**.
 - O resultado é inserido em uma **função de ativação**, que irá classificar o resultado, gerando uma saída.



Regressão com Redes Neurais

- **Revisão**
 - O **treinamento** da rede segue o seguinte algoritmo:
 1. Inicializa os pesos aleatoriamente
 2. Calcula as saídas
 3. Calcula o erro da rede
 4. Calcula novos pesos
 5. Atualiza os pesos
 6. Se o erro ainda não reduziu o suficiente, voltar ao passo 2.

Regressão com Redes Neurais

- **Revisão**

- O **treinamento** da rede segue o seguinte algoritmo:

1. Inicializa os pesos aleatoriamente
2. Calcula as saídas
3. **Calcula o erro da rede**
4. Calcula novos pesos
5. Atualiza os pesos
6. Se o erro ainda não reduziu o suficiente, voltar ao passo 2.



Também chamado de “custo”. O treinamento da rede é um problema de otimização, em que queremos minimizar o erro.

Regressão com Redes Neurais

- **Revisão**

- O **treinamento** da rede segue o seguinte algoritmo:

1. Inicializa os pesos aleatoriamente
2. Calcula as saídas
3. Calcula o erro da rede
4. **Calcula novos pesos**
5. Atualiza os pesos
6. Se o erro ainda não reduziu o suficiente, voltar ao passo 2.

O cálculo dos novos pesos pode ser feito de diversas maneiras.

Alguns exemplos:

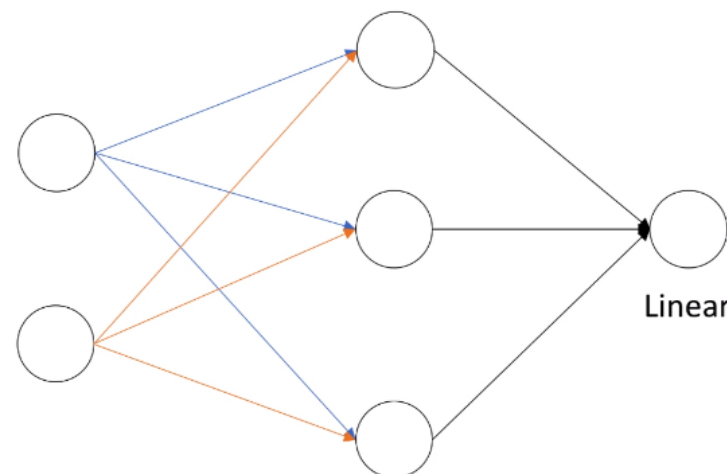
- Stochastic Gradient Descent
- Quasi-Newton methods
- Backpropagation
- Momentum
- Adam

Regressão com Redes Neurais

- **Redes Neurais para Regressão**

- A única diferença do funcionamento das Redes Neurais Multi-Perceptron em problemas de regressão está na **camada de saída**.
- Para este tipo de problema, a última camada será sempre composta por um único neurônio, com a **função de ativação linear**.

➤ Com isso, o resultado não será a separação em classes, mas um **único valor numérico**.



Regressão com Redes Neurais

- **Redes Neurais para Regressão**
 - Para regressão, as Redes Neurais precisam que os mesmos parâmetros de configuração sejam especificados:
 - **Número de camadas** escondidas
 - **Número de neurônios** nas camadas escondidas
 - Número máximo de **iterações**
 - **Tolerância**
 - **“Solver”**: Método de otimização utilizado para minimizar o erro da rede e calcular nos novos pesos.
 - **Função de ativação**: Função utilizada para dizer se um neurônio está ativado ou não.

- **Redes Neurais para Regressão**

- Vamos voltar para o problema simples do plano de saúde.
- Como vimos, neste problema é possível visualizar os dados da regressão.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

base = pd.read_csv('plano_saude.csv')

# Separando dados em previsores e classes
cols_previsores = ['idade']

cols_objetivo = ['custo']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# Separando em base de testes e treinamento (usando 25% para teste)
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
                                                                                                    objetivo,
                                                                                                    test_size=0.25,
                                                                                                    random_state=0)

# Padronização dos dados
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
previsores_treinamento = scaler.fit_transform(previsores_treinamento)
scaler_y = StandardScaler()
objetivo_treinamento = scaler_y.fit_transform(objetivo_treinamento)

# Visualização dos dados
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.title('Regressão com Redes Neurais (dados treinamento)')
plt.xlabel('idade')
plt.ylabel('custo')
```

- **Redes Neurais para Regressão**

- Vamos voltar para o problema simples do plano de saúde.
- Como vimos, neste problema é possível visualizar os dados da regressão.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

base = pd.read_csv('plano_saude.csv')

# Separando dados em previsores e classes
cols_previsores = ['idade']

cols_objetivo = ['custo']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# Separando em base de testes e treinamento (usando 25%)
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores, objetivo, test_size=0.25, random_state=42)

# Padronização dos dados
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
previsores_treinamento = scaler.fit_transform(previsores_treinamento)
scaler_y = StandardScaler()
objetivo_treinamento = scaler_y.fit_transform(objetivo_treinamento)

# Visualização dos dados
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.title('Regressão com Redes Neurais (dados treinamento)')
plt.xlabel('idade')
plt.ylabel('custo')
```

Assim como no caso da regressão com SVM, é importante fazer a padronização dos dados quando usamos as Redes Neurais para regressão.



- **Redes Neurais para Regressão**

- Vamos voltar pa

- Como vimos, ne

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np
```

```
##### Preprocessamento
```

```
base = pd.read_csv('plano_sa.csv')
```

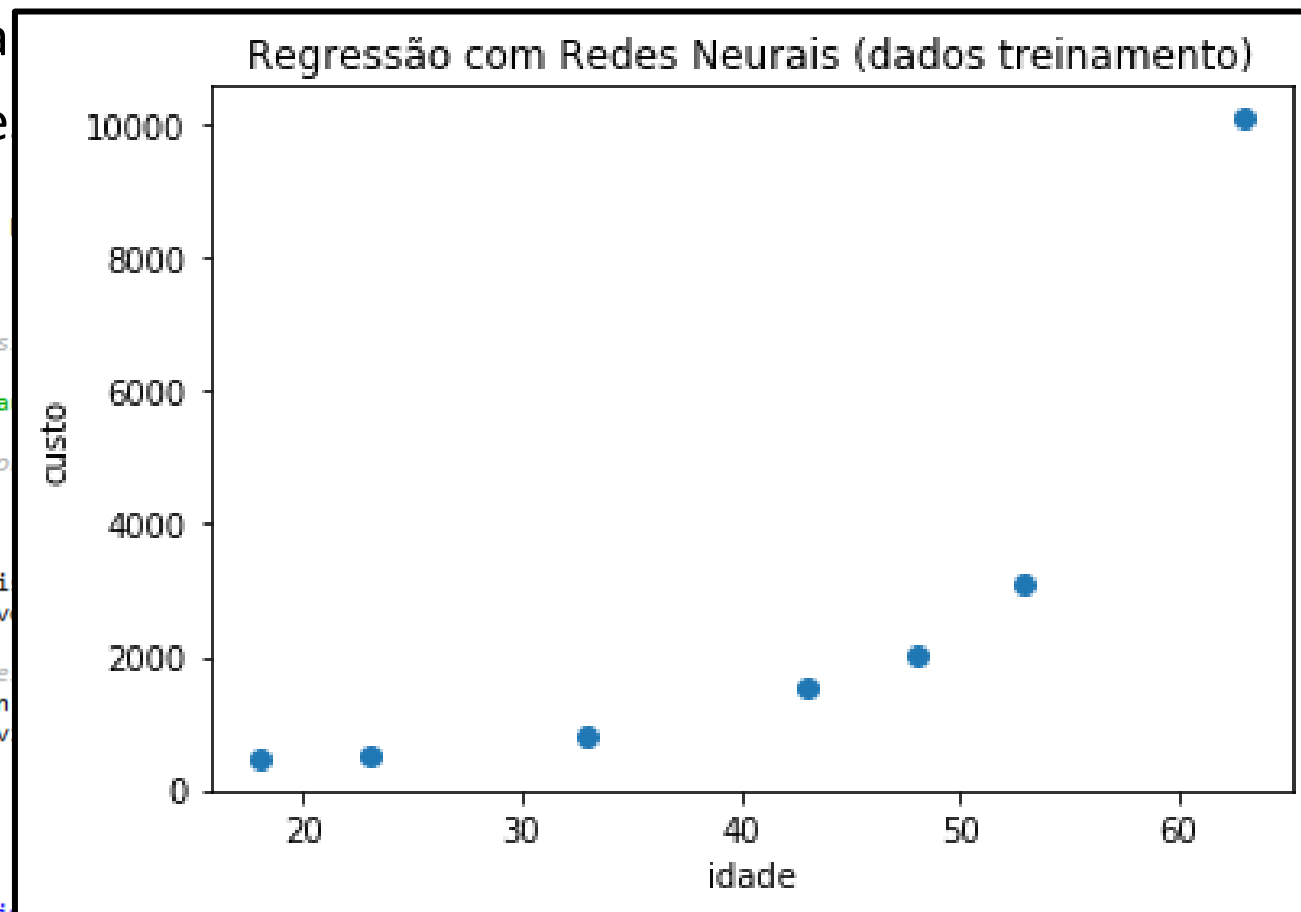
```
# Separando dados em preditores e objetivo
cols_preditores = ['idade']
```

```
cols_objetivo = ['custo']
preditores = base[cols_preditores]
objetivo = base[cols_objetivo]
```

```
# Separando em base de treinamento e teste
from sklearn.model_selection import train_test_split
preditores_treinamento, preditores_teste, objetivo_treinamento, objetivo_teste = train_test_split(preditores, objetivo, test_size=0.2, random_state=42)
```

```
# Padronização dos dados
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
preditores_treinamento = scaler.fit_transform(preditores_treinamento)
scaler_y = StandardScaler()
objetivo_treinamento = scaler_y.fit_transform(objetivo_treinamento)
```



Mesmo com a padronização, o comportamento dos dados permanece similar, mesmo que em outra escala.

- **Redes Neurais para Regressão**

- Para aplicar a Regressão com Redes Neurais, devemos fazer:

```
##### Regressão com Redes Neurais #####
```

```
from sklearn.neural_network import MLPRegressor
regressor = MLPRegressor(activation='relu',
                          max_iter=200,
                          verbose=True,
                          hidden_layer_sizes = (100),
                          random_state=0)
```

```
# Treinamento
```

```
regressor.fit(previsores_treinamento, objetivo_treinamento)
```

```
# Teste
```

```
previsoes = regressor.predict scaler.transform(previsores_teste))
previsoes = scaler_y.inverse_transform(previsoes)
```

- **Redes Neurais para Regressão**

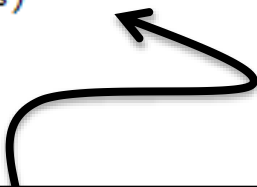
- Para aplicar a Regressão com Redes Neurais, devemos fazer:

```
##### Regressão com Redes Neurais #####
```

```
from sklearn.neural_network import MLPRegressor
regressor = MLPRegressor(activation='relu',
                        max_iter=200,
                        verbose=True,
                        hidden_layer_sizes = (100),
                        random_state=0)
```

```
# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)
```

```
# Teste
previsoes = regressor.predict(scaler.transform(previsores_teste))
previsoes = scaler_y.inverse_transform(previsoes)
```



Para ver a previsão na escala original, é necessário aplicar a transformação inversa da variável previsões, usando a mesma instancia do Scaler usado para a padronização da variável objetivo.

- **Redes Neurais para Regressão**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

```
##### Avaliação dos resultados #####

# Visualização dos dados de treinamento
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(sorted(previsores_treinamento),
          sorted(regressor.predict(previsores_treinamento)), color = 'red')
plt.title('Regressão com Redes Neurais')
plt.xlabel('Idade')
plt.ylabel('Custo')

# Visualização dos dados de teste
plt.scatter(previsores_teste, objetivo_teste)
plt.plot(sorted(previsores_teste.values), sorted(previsoes), color = 'red')
plt.title('Regressão com Redes Neurais')
plt.xlabel('Idade')
plt.ylabel('Custo')

score = metrics.r2_score(objetivo_teste, previsoes)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

- **Redes Neurais para Regressão**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

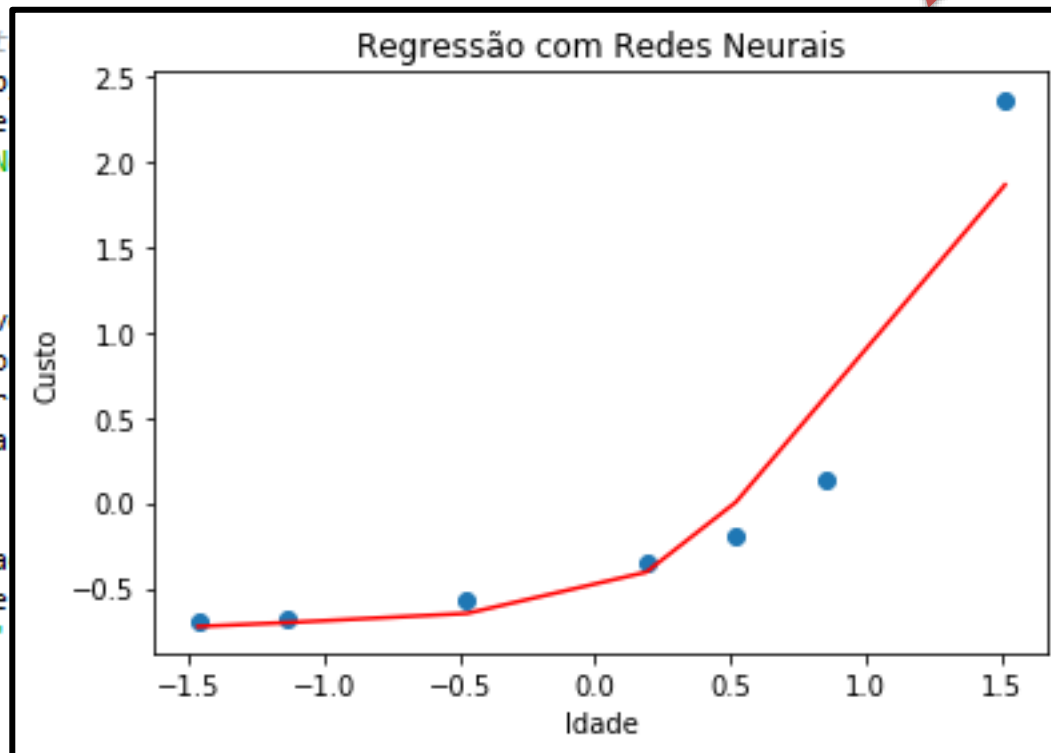
Avaliação dos resultados

```
# Visualização dos dados de treinamento
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(sorted(previsores_treinamento),
          sorted(regressor.predict(previsores_treinamento)), color = 'red')
plt.title('Regressão com Redes Neurais')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

```
# Visualização dos dados de teste
plt.scatter(previsores_teste, objetivo_teste)
plt.plot(sorted(previsores_teste), sorted(regressor.predict(previsores_teste)), color = 'red')
plt.title('Regressão com Redes Neurais')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

```
score = metrics.r2_score(objetivo_teste, regressor.predict(previsores_teste))
mae = metrics.mean_absolute_error(objetivo_teste, regressor.predict(previsores_teste))
mse = metrics.mean_squared_error(objetivo_teste, regressor.predict(previsores_teste))
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, regressor.predict(previsores_teste)))
```

```
print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```



- **Redes Neurais para Regressão**

- Para analisar os resultados, podemos plotar o gráfico das previsões:

```
##### Avaliação dos resultados #####
```

```
# Visualização dos dados de treinamento
```

```
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(sorted(previsores_treinamento),
          sorted(regressor.predict(previsores_treinamento)), color = 'red')
plt.title('Regressão com Redes Neurais')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

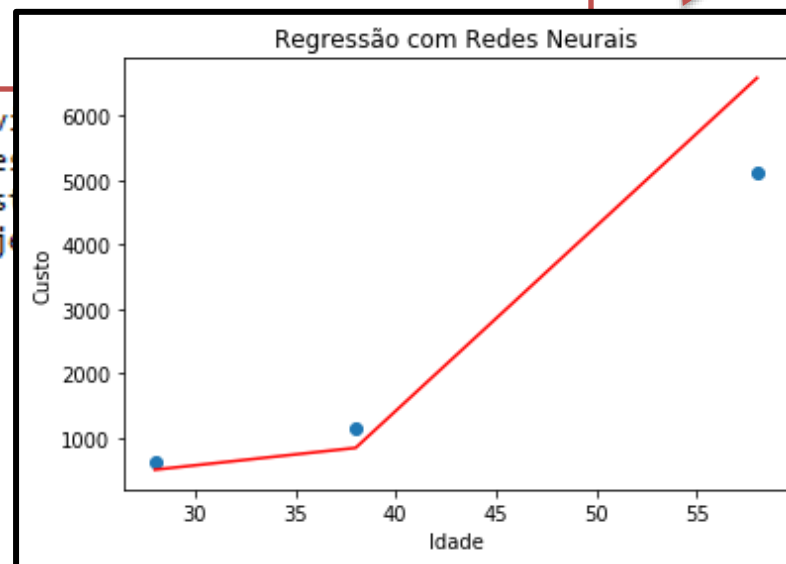
```
# Visualização dos dados de teste
```

```
plt.scatter(previsores_teste, objetivo_teste)
plt.plot(sorted(previsores_teste.values), sorted(previsoes), color = 'red')
plt.title('Regressão com Redes Neurais')
plt.xlabel('Idade')
plt.ylabel('Custo')
```

```
score = metrics.r2_score(objetivo_teste, previsoes)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
```

Os resultados na base de teste
foram:

```
Score: 0.8077356965885252
Mean Absolute Error: 635.595872736667
Mean Squared Error: 765933.986850482
Root Mean Squared Error: 875.1765461039744
```



- **Redes Neurais para Regressão**

- Voltando ao problema dos preços de imóveis, o pré-processamento fica igual.

[illegible]

- **Redes Neurais para Regressão**

- Para aplicar o SVR, ele deve ser definido como regressor, e seus parâmetros devem ser configurados.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Préprocessamento #####

scaler_y = StandardScaler()
objetivo_treinamento = scaler_y.fit_transform(objetivo_treinamento)

##### Regressão com Árvores de Decisão #####

from sklearn.neural_network import MLPRegressor
regressor = MLPRegressor(activation='relu',
                          max_iter=300,
                          verbose=True,
                          hidden_layer_sizes = (10,9),
                          random_state=0)

# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(previsores_teste)
previsoes = scaler_y.inverse_transform(previsoes)

##### Avaliação dos resultados #####

score = metrics.r2_score(objetivo_teste, previsoes)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

Assim como no SVR, os melhores resultados são obtidos quando a variável objetivo também é padronizada.

Lembrando sempre que, ao fazer isso, é necessário desfazer a padronização para fazer as previsões e calcular as métricas.

• Redes Neurais

- Para aplicar os melhores parâmetros

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np
```

```
##### Préprocessamento #####
```

```
scaler_y = StandardScaler()
objetivo_treinamento = scaler_y.fit_transform(y_train)
```

```
#####
```

```
from sklearn.neural_network import MLPRegressor
regressor = MLPRegressor()
```

```
# Treinamento
regressor.fit(previsoes_treinamento, objetivo_treinamento)
```

```
# Teste
previsoes_teste = regressor.predict(previsoes_teste)
```

```
previsoes = scaler_y.inverse_transform(previsoes)
```

```
#####
```

```
score = metrics.r2_score(y_test, previsoes)
mae = metrics.mean_absolute_error(y_test, previsoes)
mse = metrics.mean_squared_error(y_test, previsoes)
rmse = np.sqrt(mse)
```

```
print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

Vamos registrar os resultados em um arquivo único, para que possamos compará-lo com resultados futuros:

Score: 0.8937784219319959

Mean Absolute Error: 71150.68022604634

Mean Squared Error: 14110536875.079235

Root Mean Squared Error: 118787.78083237028



variável objetivo também é padronizada.

House Sales in King County, USA				
Descrição	Dados e imóveis e seus respectivos valores em um distrito dos Estados Unidos.			
Total de registros:	21613			
Total de características:	16			
Total de colunas objetivo:	1			
Resultados				
Método	Configuração	Tratamento de dados	Score %	RMSE
Regressão Linear	-	Padronização	0.69	202633.35
Regressão Polinomial	Grau 2	Padronização	0.8083	159537.82
Árvores de Decisão	max_depth=9	Padronização	0.8002	162896.46
Random Forest	n_estimators=100, max_features=10, max_depth=15	Padronização	0.8889	121506.52
SVR	kernel = 'rbf', C = 1, gamma = 'scale', epsilon = 0.1	Padronização (Total)	0.7640	177051.47
Redes Neurais	activation='relu', max_iter=300, verbose=True, hidden_layer_sizes = (10,9), random_state=0	Padronização (Total)	0.8938	118787.78

regressor, e seus

os melhores
os quando a

que, ao fazer
desfazer a
a fazer as
as métricas.

- **Redes Neurais para Regressão**

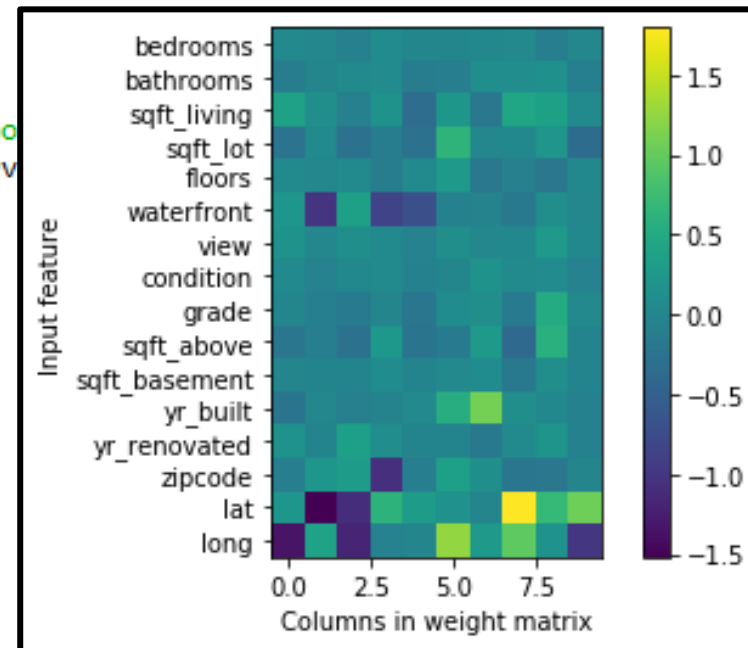
- Assim como as técnicas **Árvores de Decisão** e as **Florestas Randômicas**, é possível fazer uma análise da importância das variáveis na previsão dos valores.
- Apesar de **não ser** uma avaliação de importância tão **precisa** quanto nas outras técnicas, podemos avaliar o **peso** que cada **variável previsora** possuiu na **primeira camada** da Rede Neural.
- Para isso, basta implementar as seguintes linhas de código:

```
# Plotagem do "Mapa de calor" de uma rede neural
import matplotlib.pyplot as plt
plt.imshow(regressor.coefs_[0], interpolation='none', cmap='viridis')
plt.yticks(range(len(cols_previsores)), cols_previsores)
plt.xlabel("Columns in weight matrix")
plt.ylabel("Input feature")
plt.colorbar()
```

- **Redes Neurais para Regressão**

- Assim como as técnicas **Árvores de Decisão** e as **Florestas Randômicas**, é possível fazer uma análise da importância das variáveis na previsão dos valores.
- Apesar de **não ser** uma avaliação de importância tão **precisa** quanto nas outras técnicas, podemos avaliar o **peso** que cada **variável preditora** possuiu na **primeira camada** da Rede Neural.
- Para isso, basta implementar as seguintes linhas de código:

```
# Plotagem do "Mapa de calor" de uma rede neural
import matplotlib.pyplot as plt
plt.imshow(regressor.coefs_[0], interpolation='none')
plt.yticks(range(len(cols_previsores)), cols_prev
plt.xlabel("Columns in weight matrix")
plt.ylabel("Input feature")
plt.colorbar()
```



Avaliação de Modelos de Regressão

Avaliação de Modelos de Regressão

- **Introdução**

- Em nossas aulas você aprendeu a utilizar algumas **métricas** para avaliar o **desempenho** de um algoritmo de regressão.
- Todas elas medem, de alguma forma, a **aderência** que o **modelo treinado** possui com os dados.
- As métricas usadas são:

- R^2 -score
- Mean Absolute Percentage Error
- Mean Absolute Error
- Mean Square Error
- Root Mean Squared Error



Avaliação de Modelos de Regressão

- **Métricas**

- O **R^2 -score** (coeficiente de determinação) é definido como a proporção da variância da variável dependente que é previsível pelas variáveis independentes.

$$R^2 = 1 - \frac{\sum_{i=1}^N (x_{exp} - x_{modelo})^2}{\sum_{i=1}^N (x_{exp} - x_{médio})^2}$$

- Obs: Possui valor 1 como máximo e pode assumir valores arbitrariamente baixos.

Avaliação de Modelos de Regressão

- **Métricas**

- O **Erro Médio Percentual Absoluto** (*MAPE - Mean Absolute Percentage Error*), também conhecido como desvio percentual absoluto médio, é uma medida da precisão da previsão de um método de previsão em estatísticas.

$$MAPE = \frac{1}{N} \sum_{i=1}^N abs \left(\frac{x_{exp} - x_{modelo}}{x_{exp}} \right)$$

- Obs: O erro percentual absoluto médio é comumente na avaliação de modelos, devido à sua interpretação muito intuitiva em termos de erro relativo.

Avaliação de Modelos de Regressão

- **Métricas**

- O **Erro Médio Absoluto** (*MAE - Mean Absolute Error*) é definido como a média da soma do valor absoluto das distâncias entre os pontos experimentais e os simulados pelo modelo.

$$MAE = \frac{1}{N} \sum_{i=1}^N \text{abs}(x_{exp} - x_{modelo})$$

- Obs: Como o valor absoluto é utilizado, o erro medido pela métrica está na mesma magnitude que os dados utilizados.

Avaliação de Modelos de Regressão

- **Métricas**

- O **Erro Médio Quadrático** (*MSE - Mean Square Error*) é definido como a média da soma quadrática das distâncias entre os pontos experimentais e os simulados pelo modelo (variância).

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_{exp} - x_{modelo})^2$$

- Obs: Por ser elevado ao quadrado, uma pequena variação entre os dados tem um grande impacto na métrica.

Avaliação de Modelos de Regressão

- **Métricas**

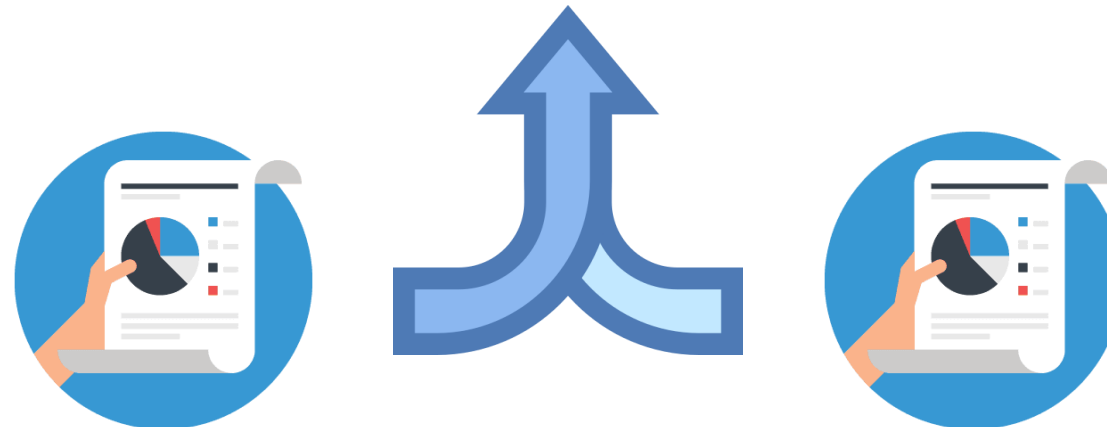
- A **Raiz do Erro Médio Quadrático** (*RMSE – Root Mean Square Error*) é definido como a raiz da média da soma quadrática das distâncias entre os pontos experimentais e os simulados pelo modelo (desvio padrão).

$$RMSE = \sqrt{MSE}$$

- Obs: Corresponde ao desvio padrão dos dados estimados com os dados reais. Assim como o MAE, a magnitude é a mesma dos dados.

Avaliação de Modelos de Regressão

- **Validação Cruzada (Cross-validation)**
 - O mesmo procedimento de validação cruzada que vimos para os problemas de classificação pode ser usado para regressão.
 - É mais **estável** e **confiável** que usar apenas a divisão em base de treinamento e testes.
 - Na validação cruzada, os dados são divididos **repetidas vezes** e diferentes modelos são treinados.



- **Construindo o script**

- Primeiramente, devemos identificar quais foram os melhores resultados que obtivemos.

House Sales in King County, USA				
Descrição	Dados e imóveis e seus respectivos valores em um distrito dos Estados Unidos.			
Total de registros:	21613			
Total de características:	16			
Total de colunas objetivo:	1			
Resultados				
Método	Configuração	Tratamento de dados	Score %	RMSE
Regressão Linear	-	Padronização	0.69	202633.35
Regressão Polinomial	Grau 2	Padronização	0.8083	159537.82
Árvores de Decisão	max_depth=9	Padronização	0.8002	162896.46
Random Forest	n_estimators=100, max_features=10, max_depth=15	Padronização	0.8889	121506.52
SVR	kernel = 'rbf', C = 1, gamma = 'scale', epsilon = 0.1	Padronização (Total)	0.7640	177051.47
Redes Neurais	activation='relu', max_iter=300, verbose=True, hidden_layer_sizes = (10,9), random_state=0	Padronização (Total)	0.8938	118787.78

- **Construindo o script**
 - Para aplicar a validação cruzada, o pré-processamento deve ser feito **sem a separação** em bases de **treinamento** e de **testes**.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

#Leitura dos dados
base = pd.read_csv('house_prices.csv')
base.describe()

# Procurando valores inconsistentes
# Não há valores inconsistentes

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Separando dados em previsores e classes
cols_previsores = ['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
                   'floors', 'waterfront', 'view', 'condition', 'grade', 'sqft_above',
                   'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode', 'lat', 'long']
# Não usarei: date, sqft_living15 e sqft_lot15

cols_objetivo = ['price']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# Transforma as variáveis categóricas em valores numéricos
# Todas as variáveis são numéricas

# Padronização dos dados
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
previsores = scaler.fit_transform(previsores)

scaler_y = StandardScaler()
objetivo = scaler_y.fit_transform(objetivo)
```

- **Construindo o script**
 - O script para fazer a **validação cruzada** fica assim, por exemplo, para a utilização das **Redes Neurais**:

```
#####  
##### Validação cruzada #####  
#####  
  
# Regressor  
from sklearn.neural_network import MLPRegressor  
  
# Divisão dos dados para validação cruzada  
from sklearn.model_selection import StratifiedKFold  
  
kfold = StratifiedKFold(n_splits = 10, shuffle = True, random_state = 3)  
scores = []  
maes = []  
mses = []  
rmse = []  
for indice_treinamento, indice_teste in kfold.split(previsores,  
                                                    np.zeros(shape=(previsores.shape[0], 1))):  
    regressor = MLPRegressor(activation='relu',  
                             max_iter=300,  
                             verbose=True,  
                             hidden_layer_sizes = (10,9),  
                             random_state=0)  
  
    # Treinamento  
    regressor.fit(previsores[indice_treinamento], objetivo[indice_treinamento,0])  
  
    previsoes = regressor.predict(previsores[indice_teste])  
    previsoes = scaler_y.inverse_transform(previsoes)  
    objetivo_escala_original = scaler_y.inverse_transform(objetivo[indice_teste,0])  
  
    score = metrics.r2_score(objetivo_escala_original, previsoes)  
    mae = metrics.mean_absolute_error(objetivo_escala_original, previsoes)  
    mse = metrics.mean_squared_error(objetivo_escala_original, previsoes)  
    rmse = np.sqrt(metrics.mean_squared_error(objetivo_escala_original, previsoes))  
  
    scores.append(score)  
    maes.append(mae)  
    mses.append(mse)  
    rmse.append(rmse)
```


- **Construindo o script**

- O script para fazer a **validação cruzada** fica assim, por exemplo, para a utilização das **Redes Neurais**:

```
#####
##### Validação cruzada #####
#####

# Regressor
from sklearn.neural_network import MLPRegressor

# Divisão dos dados para validação cruzada
from sklearn.model_selection import StratifiedKFold

kfold = StratifiedKFold(n_splits = 10, shuffle = True, random_state = 3)
scores = []
maes = []
mses = []
rmse = []
for indice_treinamento, indice_teste in kfold.split(previsores,
                                                    np.zeros(shape=(previsores.shape[0], 1))):

    regressor = MLPRegressor(activation='relu',
                             max_iter=300,
                             verbose=True,
                             hidden_layer_sizes = (10,9),
                             random_state=0)

    # Treinamento
    regressor.fit(previsores[indice_treinamento], objetivo[indice_treinamento,0])

    previsoes = regressor.predict(previsores[indice_teste])
    previsoes = scaler_y.inverse_transform(previsoes)
    objetivo_escala_original = scaler_y.inverse_transform(objetivo[indice_teste,0])

    score = metrics.r2_score(objetivo_escala_original, previsoes)
    mae = metrics.mean_absolute_error(objetivo_escala_original, previsoes)
    mse = metrics.mean_squared_error(objetivo_escala_original, previsoes)
    rmse = np.sqrt(metrics.mean_squared_error(objetivo_escala_original, previsoes))

    scores.append(score)
    maes.append(mae)
    mses.append(mse)
    rmse.append(rmse)
```

Para usar outros regressores, basta trocar nestes locais.

- **Construindo o script**

- Depois, devemos calcular os resultados médios para as métricas:

```
##### Resultado final #####  
# Métricas médias  
scores = np.asarray(scores)  
score_final_medio = scores.mean()  
score_final_desvio_padrao = scores.std()  
  
maes = np.asarray(maes)  
mae_final_medio = maes.mean()  
mae_final_desvio_padrao = maes.std()  
  
mses = np.asarray(mses)  
mse_final_medio = mses.mean()  
mse_final_desvio_padrao = mses.std()  
  
rmse = np.asarray(rmse)  
rmse_final_medio = rmse.mean()  
rmse_final_desvio_padrao = rmse.std()
```

- **Construindo o script**

- Depois, devemos calcular os resultados médios para as métricas:

```
##### Resultado final #####  
# Métricas médias  
scores = np.asarray(scores)  
score_final_medio = scores.mean()  
score_final_desvio_padrao = scores.std()  
  
maes = np.asarray(maes)  
mae_final_medio = maes.mean()  
mae_final_desvio_padrao = maes.std()  
  
mses = np.asarray(mses)  
mse_final_medio = mses.mean()  
mse_final_desvio_padrao = mses.std()  
  
rmsees = np.asarray(rmsees)  
rmse_final_medio = rmsees.mean()  
rmse_final_desvio_padrao = rmsees.std()
```

Os resultados podem variar dos obtidos antes, pois agora representam a média dos ajustes.

Devemos registrar os resultados em um novo arquivo.

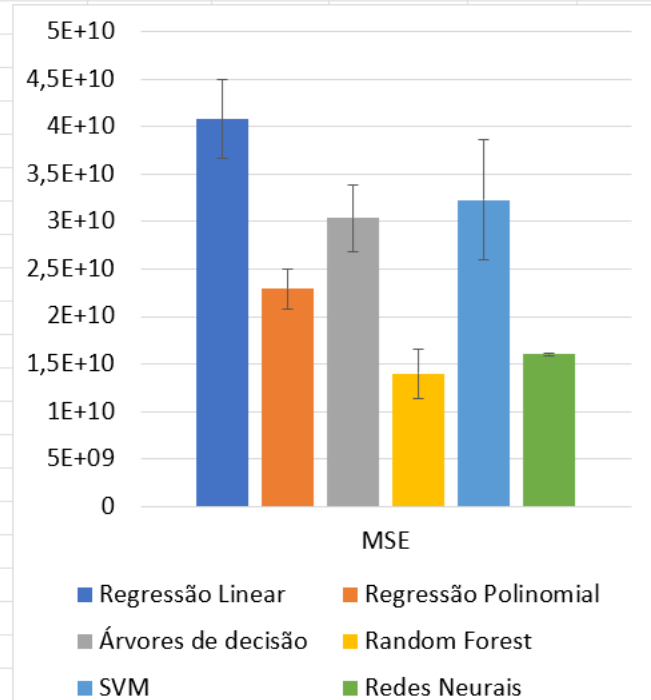
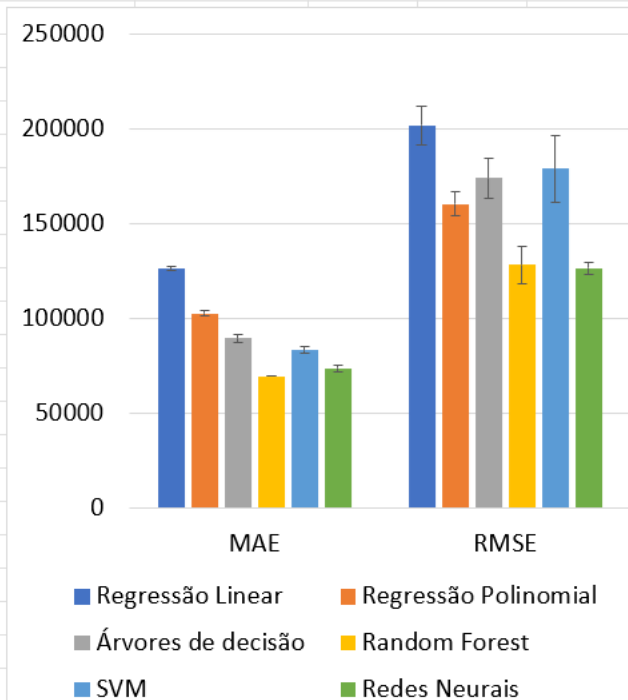
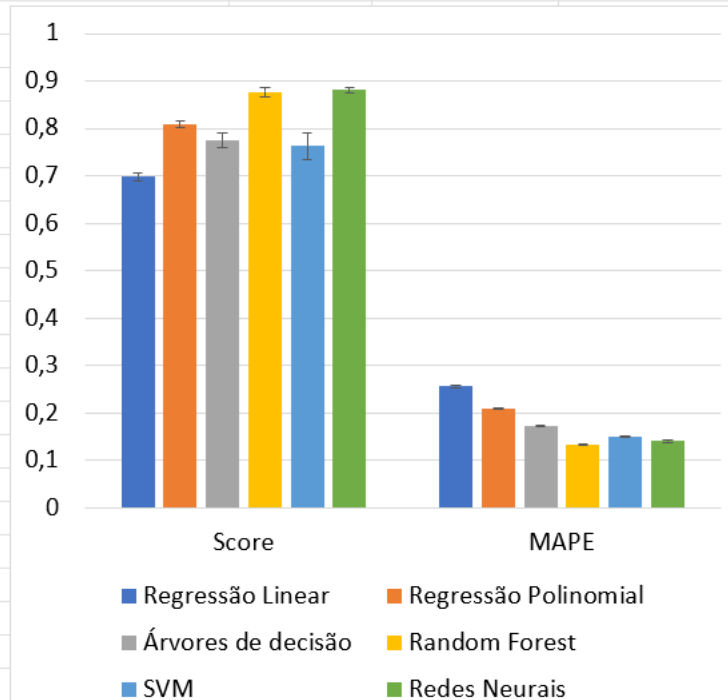
mae_final_desvio_padrao	float64	1	2142.2112551110304
mae_final_medio	float64	1	72932.06231899015
mse_final_desvio_padrao	float64	1	1410929736.9434524
mse_final_medio	float64	1	15443467420.690144
rmse_final_desvio_padrao	float64	1	5735.596148999185
rmse_final_medio	float64	1	124139.318338332
score_final_desvio_padrao	float64	1	0.010209959112285184
score_final_medio	float64	1	0.8841445320775007

- **Construindo o script**

- O novo arquivo, contendo os resultados finais para as principais configurações testadas, deve ficar assim:

Nome da Base					
	Médias				
Método	Score	MAPE	MAE	RMSE	MSE
Regressão Linear	0,6974471	0,256036499	126243,4275	201757,543	40811804822
Regressão Polinomial	0,80844674	0,208887021	102792,7468	160449,5747	22922602820
Árvores de decisão	0,77471677	0,17184424	89578,71954	174016,4955	30387452647
Random Forest	0,87747717	0,133210291	69565,89336	128428,949	13964461932
SVM	0,76220278	0,149381812	83273,01513	178879,5667	32319178783
Redes Neurais	0,880838	0,1405	73696,77841	126448,8529	16001112417

Desvios				
Score	MAPE	MAE	RMSE	MSE
0,00827	0,0032388	1030,102	10280,99	4,14E+09
0,00604	0,0013375	1448,94	6541,523	2,119E+09
0,0151	0,0016013	1910,237	10281,63	3,531E+09
0,00981	0,0018344	0,001834	9947,43	2,589E+09
0,02938	0,0018588	1884,523	17924,27	6,358E+09
0,00587	0,00294	1497,788	3435,115	86195658



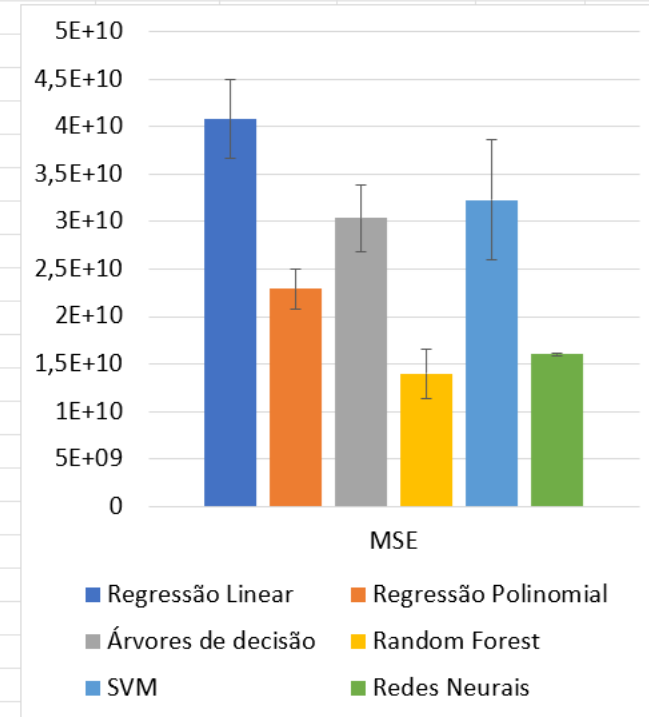
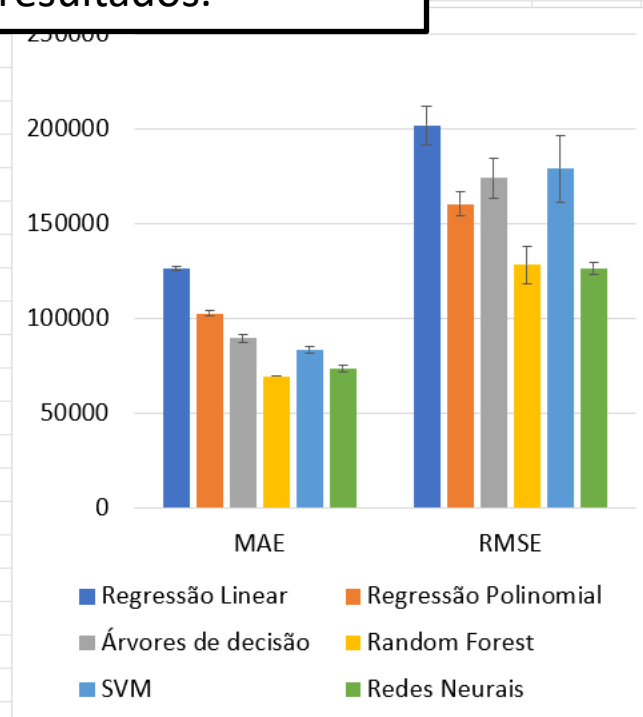
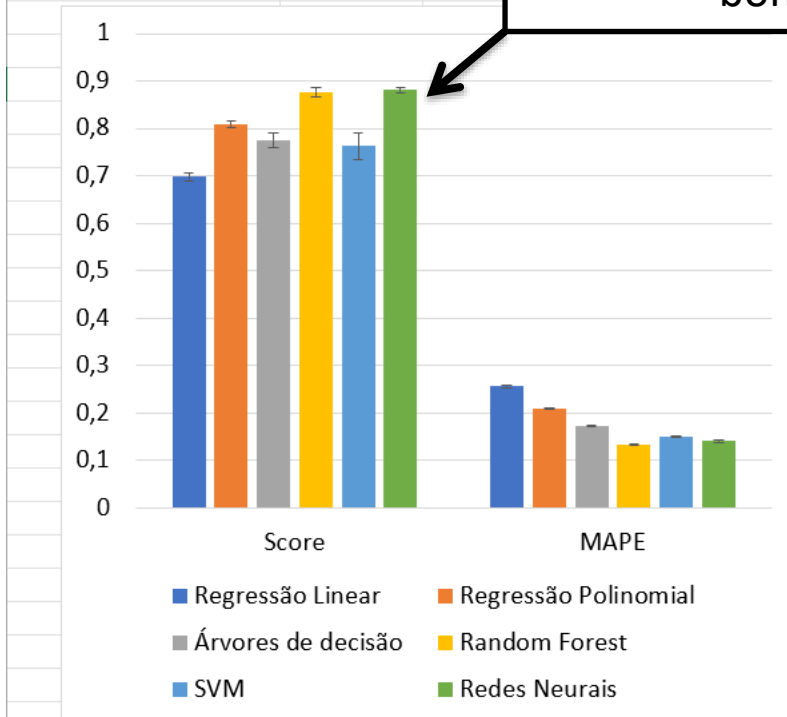
- **Construindo o script**

- O novo arquivo, contendo os resultados finais para as principais configurações testadas, deve ficar assim:

Nome da Base		
Médias		
Método	Score	MAPE
Regressão Linear	0,6974471	0,23
Regressão Polinomial	0,80844674	0,20
Árvores de decisão	0,77471677	0,19
Random Forest	0,87747717	0,13
SVM	0,76220278	0,14
Redes Neurais	0,880838	

Repare que o melhor resultado foi o das Redes Neurais. Porém, levando em conta todas as métricas, o Random Forest também alcançou bons resultados.

Desvios				
Score	MAPE	MAE	RMSE	MSE
0,00827	0,0032388	1030,102	10280,99	4,14E+09
0,00604	0,0013375	1448,94	6541,523	2,119E+09
0,0151	0,0016013	1910,237	10281,63	3,531E+09
0,00981	0,0018344	0,001834	9947,43	2,589E+09
0,02938	0,0018588	1884,523	17924,27	6,358E+09
0,00587	0,00294	1497,788	3435,115	86195658



Regressão com Redes Neurais

FIM